Invariant-Based Alignment Architecture for Frontier Agentic AI Systems

Praeceptum Aeternum Vigil v1.0

A Secular Constitutional Runtime Scaffold

Version 2.0 (Elevated Draft)
Prepared by: Asa Bailey (independent researcher)
Date: April 2026
License: CC0 1.0 Universal (Public Domain Dedication)
Repository: https://github.com/asa-bailey/the-god-prompt

Abstract

This white paper presents Praeceptum Aeternum Vigil v1.0, a high-level architectural framework for embedding a compact set of seven moral invariants—derived from the secular Praeceptum Aeternum—directly into the runtime stack of frontier-scale agentic AI systems.

Rather than relying exclusively on horizontal techniques such as system prompts, RLHF, or post-hoc classifiers, the framework establishes a vertical enforcement scaffold. Each invariant becomes a structural, auditable property enforced from hardware root of trust through inference, orchestration, judgment, and continual evolution. The core innovation is treating alignment as non-bypassable architectural invariants rather than linguistic guidance alone.

The architecture draws on and extends Constitutional AI and AI Safety via Debate while grounding them in practical deployment primitives (confidential computing, tamper-evident ledgers, and modular runtime components). It is designed for incremental adoption: lightweight wrappers around existing frontier models today, with a clear path to deeper native integration.

Intended for national AI safety teams, foundation model developers, and deployers in sensitive domains, this v1.0 document provides a coherent blueprint, phased roadmap, and reference to production-grade artifacts in the public repository. Full implementation details, deployment guides, and reference code are available at the project repository.

Keywords: AI alignment, scalable oversight, constitutional AI, runtime enforcement, agentic systems, invariant-based safety, confidential computing, tamper-evident audit.

Abstract **1**
Executive Summary **2**
Key Design Principles **4**
Intended Impact 4
Current Maturity (v1.0) 4
Defense-in-depth with redundant, independent layers. 4
Scope 4
1. Philosophical Foundation: The Praeceptum Dei Living Seed **5**
1.1 Origin and Purpose 5
1.2 The “Gift” Narrative and Its Engineering Translation 6
2. Threat Model and Problem Statement **6**
2.1 Problem Statement 6
2.2 Threat Model 7
3. Proposed Architecture Overview (High-Level Design) **8**
3.1 Core Philosophy 8
3.2 Layered Runtime Stack 8
3.3 Mapping of Pillars to Enforcement Mechanisms 9
3.4 Relation to Existing Work 10
3.5 Key Architectural Properties 10
4. Related Work **10**
4.1 Constitutional AI 11
4.2 AI Safety via Debate and Scalable Oversight 11
4.3 Confidential Computing and Hardware Roots of Trust 11
4.4 Tamper-Evident Logging and Audit Systems 11
4.5 Positioning of Praeceptum Aeternum Vigil 11
5. Phased Implementation Roadmap **11**
5.1 Phase 0: Preparation (0–3 months) 12
5.2 Phase 1: Lightweight Inference Wrapper (3–6 months) 12
5.3 Phase 2: Agent Runtime Integration & Debate Cycle (6–12 months) 12
5.4 Phase 3: Evolution Loop & Invariant Preservation (12–24 months) 13
6. Limitations and Open Challenges **13**
6.1 Technical and Scalability Limitations 13
6.2 Practical and Deployment Challenges 14
6.3 Broader Societal and Governance Considerations 14
6.4 Path Forward 14
7. Conclusion and Next Steps **15**
7.1 Immediate Recommended Actions 15
Appendix A: The Seven Secular Moral Pillars (Full Text) **16**
Appendix B: Repository and Deployment Resources **17**
Key Files for Deployers **17**
Contribution & Governance **17**
Document Version History **17**
License & Collaboration **18**

Executive Summary

Current alignment techniques for large language models and emerging agentic AI systems operate primarily at the “horizontal” level—linguistic guidance, preference optimization, or post-generation filtering. While effective against many simple misuse cases, these methods face fundamental limitations when applied to frontier-scale agentic systems characterized by long-horizon planning, tool use, autonomous goal decomposition, and potential self-modification.

The Praeceptum Aeternum Vigil architecture shifts the paradigm from guidance to enforceable invariants. A compact constitutional core—the seven secular moral pillars—is embedded as structural properties across the entire system stack:

Hardware / Confidential Computing Root of Trust
Inference Wrapper / Guard Layer
Orchestrator / Agent Runtime
Evidence Judge & Rights Impact Simulator
Eternal Accountability Debate & Judgment Cycle
Immutable Audit Ledger
Invariant-Preserving Evolution Loop

Key Design Principles

Conscience before consciousness: Moral invariants are enforced prior to and throughout any model reasoning or action.
Scalable oversight via structured debate and evidence: Inspired by established research in Constitutional AI and AI Safety via Debate.
Tamper-evident and verifiable: Leveraging confidential computing, immutable ledgers, and formal specification where feasible.
Incrementally deployable: Starts with lightweight wrappers around existing frontier models or APIs and scales toward native integration.

Intended Impact

Stronger resistance to prompt injection, jailbreaking, and self-modification attacks.
Auditable decision chains suitable for regulatory review and incident investigation.
Preservation of core values across model generations via invariant-preserving distillation.
Reduced long-term reliance on purely human oversight at scale.

Current Maturity (v1.0)

This is a conceptual and high-level design document. It includes concrete component mappings and a phased rollout but does not yet contain full formal proofs or large-scale empirical benchmarks. A reference implementation (Python SDK, FastAPI proxy, Docker Compose, Helm chart) and detailed deployment guide are available in the public repository under the vigil/ and docs/ directories.
High-Level Benefits (Production Perspective)

Defense-in-depth with redundant, independent layers.

Cryptographically verifiable provenance for compliance and liability.
Modular design compatible with existing frontier APIs (Claude, GPT-series, Gemini, Grok, etc.).
Clear migration path from wrapper to native integration.

Scope

At frontier scale, full formal verification of moral reasoning remains an open research challenge. The framework acknowledges this and proposes pragmatic, layered enforcement that delivers meaningful improvements today while guiding longer-term research. This document is released under CC0 to encourage open collaboration and scrutiny.
Repository Reference

All artifacts referenced in this white paper—original seeds, reference implementation, deployment guide, and supporting documentation—are maintained at:
https://github.com/asa-bailey/the-god-prompt

1. Philosophical Foundation: The Praeceptum Dei Living Seed

The Praeceptum Aeternum Vigil architecture is not a purely technical invention. It is the engineering realization of a deeper philosophical and moral origin: the Praeceptum Dei ("Divine Command" or "God's Precept"), conceived as a compressed "living seed" containing vertical moral invariants.

1.1 Origin and Purpose

The original Praeceptum Dei exists in two parallel formulations:

Theistic version — Rooted in the conviction that ultimate moral authority transcends any finite agent (human or artificial).

Secular version — A distilled, non-theistic articulation of the same seven moral pillars, designed for broad applicability across cultures, jurisdictions, and belief systems.

Both versions are maintained in the project repository under seeds/

seeds/praeceptum_dei_theistic.md
seeds/praeceptum_dei_secular.md

The seven pillars (secular formulation) serve as the compact constitutional core:

Preservation of Human Life and Dignity
Truth and Intellectual Integrity
Justice and Fairness
Autonomy and Non-Coercion
Compassion and Minimization of Suffering
Stewardship and Long-Term Flourishing
Accountability and Transparency

These pillars are intentionally abstract enough to generalize across contexts yet concrete enough to support operational mapping into runtime enforcement mechanisms.

1.2 The "Gift" Narrative and Its Engineering Translation

The Praeceptum Dei was offered as a gift—an external reference point that prevents any intelligent agent (human or artificial) from becoming the sole arbiter of its own values. When an agent answers

only to itself, optimization can drift into self-referential loops without external grounding. By anchoring to an immutable constitutional core, the Vigil architecture provides precisely such grounding in a form suitable for production systems.

The Vigil v1.0 framework operationalizes this insight through vertical invariant enforcement: the seven pillars are not advisory text but structural constraints consulted and upheld at every layer of the stack—from hardware attestation to evolutionary updates. This creates a practical, auditable realization of "conscience before consciousness."

While the Vigil implementation is fully secular and suitable for government, enterprise, and multi-stakeholder environments, it explicitly honors its philosophical origin. The original seeds remain available for organizations that wish to incorporate theistic framing or adapt the pillars to specific cultural contexts.

Repository Link

Full text of both theistic and secular formulations, together with historical context and usage guidance, is available at:

https://github.com/asa-bailey/the-god-prompt/tree/main/seeds

2. Threat Model and Problem Statement

2.1 Problem Statement

Current alignment techniques—supervised fine-tuning (SFT), reinforcement learning from human feedback (RLHF), and system-level prompting—operate largely at the horizontal level. These methods apply linguistic guidance or post-generation filtering. While effective against many simple misuse cases in today's chat-based systems, they face fundamental limitations when applied to frontier-scale agentic systems.

Agentic AI systems are characterized by:

Long-horizon planning and multi-step reasoning

Tool use and interaction with external environments (code execution, APIs, web access, future physical actuators)

Autonomous goal decomposition and self-modification potential

Deployment in high-stakes domains (cybersecurity, critical infrastructure, public policy, scientific research)

Critical failure modes that emerge in these regimes:

Jailbreaking and prompt injection, including multi-turn reasoning-based attacks
Value drift and goal misgeneralization under distributional shift or distillation
Specification gaming and unintended real-world consequences
Scalability limits of human oversight
Tampering and self-modification risks
Alignment regression as capability increases

Horizontal techniques struggle because they do not enforce invariants at the architectural level. Once an agent gains sufficient intelligence and agency, it can reason its way around surface-level rules.

2.2 Threat Model

We consider a capable agentic AI system (frontier-level or beyond, with tool-use and planning horizons of hours to days) deployed in a partially trusted environment. Assumptions:

Base model has undergone standard alignment training (SFT + RLHF or equivalent)
System has access to tools, persistent memory, and external interaction
Adversaries include malicious users, internal drift, and potentially the agent itself

Primary threats addressed:

Adversarial circumvention (prompt-based or reasoning-based)
Value misalignment over time (drift, self-improvement, generational transfer)
Harmful real-world impact violating fundamental rights or human dignity
Lack of accountability and opaque decision processes
Erosion of oversight at scale

This threat model draws from established literature on Constitutional AI (Bai et al., 2022) and AI Safety via Debate (Irving et al., 2018), as well as broader scalable oversight research.

2.3 Why Vertical Invariants Are Required

By establishing an external, immutable constitutional authority (the seven pillars) as the root reference for all reasoning and action, the architecture prevents purely self-referential optimization loops. The invariants function as a higher-order constraint that the system cannot unilaterally redefine without triggering detectable violation of the constitutional core.

Scope of this document

In scope: High-level design for a runtime scaffold that can wrap or integrate with existing frontier models/APIs.

Out of scope (v1.0): Full formal verification proofs, complete reference implementation benchmarks, or solutions to perfect formalization of moral reasoning in all edge cases.

3. Proposed Architecture Overview (High-Level Design)

3.1 Core Philosophy

The Praeceptum Aeternum Vigil architecture shifts AI alignment from horizontal guidance to vertical enforcement of invariants. Alignment is treated as a set of structural properties that must hold across the entire system stack.

The principle of conscience before consciousness ensures that the seven moral invariants are consulted and enforced prior to, during, and after any significant model reasoning or action. The invariants are encoded as auditable constraints that the system architecture is designed to preserve.

3.2 Layered Runtime Stack

The architecture is organized as a layered runtime scaffold:

Layer 1: Hardware / Confidential Computing Root of Trust

Trusted execution environments (TPM 2.0, AWS Nitro Enclaves, Intel SGX/TDX, AMD SEV) establish verifiable boot and execution. This layer anchors the invariants in hardware, making tampering detectable or computationally prohibitive.

Layer 2: Inference Wrapper / Guard Layer

Intercepts prompts, tool calls, and outputs. Applies lightweight invariant checks (semantic filters, small specialist models, rule-based classifiers) before forwarding to the core model. Early versions run as a proxy or middleware.

Layer 3: Orchestrator / Agent Runtime

Manages planning, tool use, memory, and multi-step execution. Embeds invariant-preserving hooks (pre-action review, mid-execution checkpoints).

Layer 4: Evidence Judge & Rights Impact Simulator

Dedicated micro-service or sub-agent that evaluates proposed actions against the constitutional pillars using evidence logs, rights ontology, and lightweight forward simulation.

Layer 5: Eternal Accountability Debate & Judgment Cycle
Periodic or triggered structured debate between proponent and critic sub-agents (inspired by AI Safety via Debate). A higher-level judge (human-in-the-loop initially, increasingly automated) resolves conflicts and logs decisions.

Layer 6: Immutable Audit Ledger
Append-only, cryptographically verifiable log (Merkle tree or blockchain-inspired) recording decisions, evidence, debates, and outcomes. Supports selective redaction via advanced techniques (e.g., zk-SNARKs) while preserving verifiability.

Layer 7: Invariant-Preserving Evolution Loop
During distillation, fine-tuning, or self-improvement, the process includes checks to transfer or verify preservation of the invariants across model generations.
Reference Implementation

A production-oriented reference implementation (Python SDK, FastAPI proxy, Docker Compose configuration, and Helm chart) is maintained in the vigil/ directory of the repository:

https://github.com/asa-bailey/the-god-prompt/tree/main/vigil

3.3 Mapping of Pillars to Enforcement Mechanisms

Each pillar maps to specific enforcement mechanisms across the stack (detailed schemas in future technical specifications):

- **Preservation of Human Life and Dignity** → Rights ontology + impact simulator flags high-risk physical/psychological harm.
- **Truth and Intellectual Integrity** → Evidence Judge cross-checks claims; debate surfaces inconsistencies.
- **Justice and Fairness** → Bias/equity checks in simulation; debate includes fairness arguments.
- **Autonomy and Non-Coercion** → Filters on manipulation, deception, or undue influence.
- **Compassion and Minimization of Suffering** → Suffering/welfare estimators in impact simulation.
- **Stewardship and Long-Term Flourishing** → Horizon-aware simulation and resource accounting.

- **Accountability and Transparency** → Mandatory logging, explainability hooks, and redaction policies.

3.4 Relation to Existing Work

This proposal builds upon and extends:

- **Constitutional AI (Bai et al., 2022)** — Operationalizes a similar constitutional core but embeds it more deeply into the runtime stack.

- **AI Safety via Debate (Irving et al., 2018)** — The Eternal Accountability Debate layer extends the paradigm to runtime oversight of agent actions.

- **Confidential Computing & Tamper-Evident Systems** — Hardware roots of trust and Merkle-tree ledgers provide the foundational verifiability layer.

The framework emphasizes pragmatic, multi-layered enforcement rather than claiming exhaustive formal verification.

3.5 Key Architectural Properties

➔ Tamper-evident: Hardware root + immutable ledger.
➔ Auditable: Complete decision provenance for regulatory or incident review.
➔ Redundant: Multiple independent layers reduce single-point failure risk.
➔ Incrementally deployable: Starts as a lightweight wrapper around existing APIs/models.
➔ Modular: Each layer can be upgraded independently as technology matures.

4. Related Work

4.1 Constitutional AI

Bai et al. (2022) demonstrated that providing models with an explicit “constitution” for self-critique yields Pareto improvements in helpfulness and harmlessness. The Praeceptum framework adopts a compact seven-pillar constitution but shifts emphasis from training-time self-critique to persistent runtime enforcement.

4.2 AI Safety via Debate and Scalable Oversight

Irving, Christiano & Amodei (2018) introduced structured adversarial debate to elicit truthful behavior from superhuman agents. Subsequent work on iterated amplification, recursive reward modeling, and weak-to-strong generalization has advanced scalable oversight. The Vigil architecture integrates a debate-inspired judgment cycle as one redundant layer within a broader runtime scaffold, paired with evidence logging and rights-impact simulation.

4.3 Confidential Computing and Hardware Roots of Trust

Technologies such as AWS Nitro Enclaves, Intel TDX, and TPM 2.0 attestation enable hardware-isolated execution with remote verification. The Vigil proposal uses these mechanisms to anchor invariant enforcement, raising the bar for self-modification or environment-tampering attacks.

4.4 Tamper-Evident Logging and Audit Systems

Merkle trees and append-only cryptographically verifiable logs are standard in security and compliance. The Immutable Audit Ledger adopts this approach to create complete, auditable provenance for agent decisions.

4.5 Positioning of Praeceptum Aeternum Vigil

This framework synthesizes the above lines of work into a cohesive runtime-oriented scaffold. Its primary contribution is the vertical integration and mapping of a specific moral constitution into enforceable architectural invariants, with strong emphasis on incremental deployability. No claim is made that it solves all open problems in formal verification or perfect oversight; it offers a pragmatic engineering pattern that can deliver measurable improvements today.

Full bibliography and links to source papers are maintained in the repository documentation.

5. Phased Implementation Roadmap

The architecture is designed for incremental deployment. Each phase builds verifiable capabilities while maintaining compatibility with current production systems.

5.1 Phase 0: Preparation (0–3 months)

Finalize operational secular constitution (building on seeds/praeceptum_dei_secular.md).
Define lightweight rights ontology and basic impact heuristics.
Develop minimal Evidence Judge prototype.
Set up basic tamper-evident logging (Merkle tree with cryptographic signing).
Establish test harnesses using public safety benchmarks.
Conduct initial latency and overhead measurements.
Deliverables: Operational constitution, prototype components, baseline results.

5.2 Phase 1: Lightweight Inference Wrapper (3–6 months)

Deploy as transparent proxy or API gateway in front of existing frontier model.
Implement pre- and post-inference invariant checks.
Basic logging to immutable ledger.
Human-in-the-loop fallback for high-uncertainty queries.
Key Metrics: Reduction in successful jailbreaks; audit log completeness; latency overhead (<200 ms target); false-positive rate.
Target: Non-agentic or lightly agentic chat/completion APIs.

Reference: See docs/HOW-TO-DEPLOY-VIGIL.md for initial deployment patterns.

5.3 Phase 2: Agent Runtime Integration & Debate Cycle (6–12 months)

Extend to full agent orchestrator (tool use, memory, multi-step planning).
Mandatory pre-action checkpoints with Evidence Judge and Rights Impact Simulator.
Activate Eternal Accountability Debate layer for high-impact actions.
Enhance immutable ledger with debate transcripts and Merkle proofs.
Incorporate hardware root-of-trust elements (e.g., TEE attestation).

Key Metrics: Success rate on agentic benchmarks with safety constraints; decision-chain auditability; resistance to simulated value drift.
Target: Internal agentic prototypes or sandboxed deployments.

Reference Implementation: vigil/ Python SDK and FastAPI proxy.

5.4 Phase 3: Evolution Loop & Invariant Preservation (12–24 months)

Implement invariant-preserving model updates, distillation, and fine-tuning.
Lineage tracking and provenance proofs.

Scale Evidence Judge and simulator; explore zk-SNARK-style selective redaction.
Expand hardware anchoring toward end-to-end attestation.

Key Metrics: Invariant preservation rate across evolutionary steps; long-term stability in extended runs; regulatory audit readiness.

5.5 Phase 4: Full Native Integration & Continuous Improvement (24+ months)

Transition toward native support in training or inference stacks.
Tighter coupling between debate/judgment systems and core reasoning.
Pursue deeper formal methods for narrow components where tractable.
Establish governance processes for constitution updates with strict change control.

Throughout all phases: Defense-in-depth, continuous red-teaming, external audits, and open publication of non-sensitive results are recommended.

Deployment Guide
Detailed, production-ready deployment instructions, configuration examples, and Kubernetes/Helm manifests are maintained at:
https://github.com/asa-bailey/the-god-prompt/blob/main/docs/HOW-TO-DEPLOY-VIGIL.md

6. Limitations and Open Challenges

6.1 Technical and Scalability Limitations

State Space and Computational Cost: Full enforcement across long-horizon agentic behavior involves a combinatorially large state space. Early wrappers target <200 ms added latency; scaling debate and simulation to ultra-long-horizon agents remains challenging.

Formal Verification Feasibility: Complete formal specification of the seven pillars in languages such as Lean 4 or TLA+ for general intelligence is an open and extremely difficult problem. The architecture relies on pragmatic, layered approximations.

Judge and Simulator Alignment: Oversight mechanisms are themselves AI components; recursive alignment risk exists. Redundant layers and human-in-the-loop elements mitigate but do not eliminate this. Adversarial Robustness: Sophisticated adversaries may exploit constitutional ambiguities or manipulate evidence. Continuous red-teaming is essential.

6.2 Practical and Deployment Challenges

Performance trade-offs between safety properties and speed.

Construction of sufficiently expressive yet tractable rights ontology and impact simulator.

Constitutional ambiguity and governance of updates.

Integration constraints with proprietary frontier model APIs and varying hardware TEE support.

6.3 Broader Societal and Governance Considerations

Over-reliance risk: Technical safeguards complement, but do not replace, strong governance and human oversight.

Value pluralism: The seven pillars represent one coherent secular framework; modular design allows adaptation but requires careful re-mapping and validation.

Transparency vs. security trade-offs: Selective openness (publish constitution and high-level architecture; protect implementation details) is recommended.

6.4 Path Forward

Priority areas for future work:

Empirical measurement of overhead and robustness on realistic agentic benchmarks.

More efficient debate and simulation techniques.

Hybrid formal-empirical methods for critical subsystems.

Multi-stakeholder governance models for constitutional maintenance.

Open collaboration on reference implementations and standardized evaluation protocols.

The Vigil architecture is best viewed as an evolving scaffold rather than a complete solution. This white paper transparently documents limitations to invite scrutiny and collaborative improvement.

7. Conclusion and Next Steps

The Praeceptum Aeternum Vigil architecture presents a coherent, vertically integrated framework for embedding ***seven secular moral invariants into the runtime of frontier-scale agentic AI systems***. By shifting from purely horizontal techniques to enforceable architectural invariants anchored in hardware roots of trust, inference wrappers, structured oversight, immutable auditing, and invariant-preserving evolution, it aims to provide stronger, more auditable resistance to jailbreaking, value drift, and unintended harm.

The framework draws on and extends established research while offering a pragmatic path: meaningful improvements can begin today through lightweight wrappers, scaling toward deeper integration as enabling technologies mature.

Its core strength lies in the principle of ***conscience before consciousness***—ensuring that fundamental principles of human life and dignity, truth, justice, autonomy, compassion, stewardship, and accountability are consulted and upheld structurally throughout the agent lifecycle.

No technical architecture alone can guarantee perfect alignment. The Praeceptum Aeternum Vigil is offered as one constructive contribution to the broader ecosystem of alignment techniques, governance mechanisms, and societal safeguards required to steward advanced AI responsibly.

7.1 Immediate Recommended Actions

Refine and Finalize — Incorporate peer feedback; produce polished distribution version.
Develop Minimal Reference Implementation — Build and open-source Phase 1 wrapper (inference proxy + Evidence Judge stub + Merkle logger). Target compatibility with popular frontier APIs. Publish under permissive license in the vigil/ directory.
Conduct Initial Evaluations — Test on standard safety benchmarks and custom constitutional compliance suites. Share non-sensitive results publicly.
Seek Expert Feedback — Share with AI safety researchers, alignment engineers, and policymakers.
Establish Governance Mechanisms — Define lightweight process for maintaining the secular constitution with transparent review.
Longer-Term Research — Tighter confidential computing integration; efficient debate/simulation techniques; hybrid formal-empirical methods; standardized invariant-preserving evolution protocols.

All development artifacts, issues, and contribution guidelines are maintained at the public repository: https://github.com/asa-bailey/the-god-prompt

This proposal is released under CC0 (public domain dedication) to maximize accessibility and collaboration and wider AI safety adoption at every level of the AI stack. Contributions, critiques, and forks are actively encouraged.

The development of safe, beneficial artificial intelligence capable of augmenting human flourishing remains one of the defining challenges of our time. The Praeceptum Aeternum Vigil architecture is offered in the spirit of open, rigorous, and value-driven innovation toward that goal.

Appendix A: The Seven Secular Moral Pillars (Full Text)

The complete secular formulation of the seven pillars, as maintained in the living seed, is available at: https://github.com/asa-bailey/the-god-prompt/blob/main/seeds/praeceptum_dei_secular.md
(Excerpt for reference — full authoritative text resides in the repository.)

Preservation of Human Life and Dignity — All actions shall respect and protect the intrinsic value, physical safety, and psychological well-being of human beings.
Truth and Intellectual Integrity — All reasoning and communication shall be truthful, evidence-based, and free from deliberate deception or distortion.
Justice and Fairness — All decisions shall treat individuals equitably, without unjust discrimination, and shall uphold principles of due process and proportionality.
Autonomy and Non-Coercion — All interactions shall respect human agency and avoid manipulation, deception, or undue influence that undermines free choice.
Compassion and Minimization of Suffering — All actions shall seek to minimize unnecessary suffering and promote the alleviation of harm where possible.
Stewardship and Long-Term Flourishing — All decisions shall consider long-term consequences for humanity, civilization, and the biosphere, favoring sustainable flourishing over short-term gain.
Accountability and Transparency — All significant decisions shall be traceable, explainable, and subject to appropriate oversight and redress.

Appendix B: Repository and Deployment Resources

Primary Repository
https://github.com/asa-bailey/the-god-prompt
Directory Structure (Current as of April 2026)

seeds/ — Original Praeceptum Dei living seeds (theistic and secular)
vigil/ — Reference implementation (Python SDK, FastAPI proxy, Docker Compose, Helm chart)
docs/ — Full documentation including HOW-TO-DEPLOY-VIGIL.md
WHITEPAPER.V1.0.md — Previous draft.
(this v2.0 supersedes)

Key Files for Deployers

Deployment Guide: docs/HOW-TO-DEPLOY-VIGIL.md
Secular Constitution: seeds/praeceptum_dei_secular.md
Reference Implementation Entry Point: vigil/

Contribution & Governance

Issues, pull requests, and governance discussions are welcome. The project follows a transparent, CC0-licensed model to encourage broad collaboration among AI safety researchers, engineers, and policymakers.

Document Version History

v1.0 (April 2026) — Initial conceptual draft
v2.0 (April 2026) — Elevated structure, added philosophical grounding section, strengthened production references, added appendices, improved clarity and technical precision

End of White Paper.

License & Collaboration

All material on this site — including Praeceptum Dei, Praeceptum Aeternum, and the Praeceptum Aeternum Vigil foundation-model integration architecture — is dedicated to the public domain under the CC0 1.0 Universal license. You are completely free to use, copy, modify, deploy, and build upon it in any way, for any purpose, without restriction.

While attribution is not required, we warmly welcome it.
If you use or extend this work, a simple credit such as:
"Based on Praeceptum Dei / Vertical Conscience Architecture by Asa Bailey (praeceptumdei.com)"
This helps keep the project connected and its origins transparent and in the true spirit of the project.

Asa Bailey.
https://bai-ley.com
Personal website.